Cartera: el pool, el universo, las parejas y la búsqueda de fondeo

AlgoProject — manual de uso. Actualizado el 2026-10-04 con `tools/manual.py`.

1. 77. Cartera · el pool de desarrollo — qué estrategias pueden entrar y cómo archivarlas
2. 78. Cartera · el universo y las parejas — la equity de cada estrategia, y qué parejas pueden ir juntas
3. 79. Cartera · la búsqueda de fondeo — qué cartera tiene más probabilidad de aprobar un plan
4. 87. Cartera · la etapa B y el veredicto de fondeo — ¿aguanta la cartera elegida un Monte Carlo, y fuera de muestra?
5. 88. La caja del fondeo — lo que sale de tu banco contra lo que vuelve, plan por plan
6. 96. Cartera: la búsqueda por rangos — qué estrategias del pool van juntas, elegidas en build

77. Cartera · el pool de desarrollo — qué estrategias pueden entrar y cómo

archivarlas

Qué pregunta responde

¿Qué estrategias de tus proyectos pueden entrar en el pool con el que se desarrolla el motor de carteras, y con qué orden exacto se archiva cada una? El comando lee los veredictos que ya dejaron los estudios de cada paso y te imprime, estrategia por estrategia, la línea de `core.archive` que la congela. **No archiva nada**: tú apruebas la lista y lanzas las líneas que quieras.

Es el hito M0 del plan de carteras (`portfolio/PLAN.md`). Las carteras de este pool **no son operables**: son para desarrollar y probar el motor hasta que exista el pool validado del paso 26.

Cuándo lo usas, y cuándo no

Úsalo cuando un run del workflow ha terminado y quieres llevar sus estrategias al archivo para construir carteras con ellas. **No** lo uses mientras SQX está corriendo ese proyecto: la lista sale igual, pero el archivo se niega a congelar una estrategia de un proyecto que SQX tiene en marcha.

Antes de empezar

Nada. Solo lee `AlgoData/reports/` y `AlgoData/strategyPermutations/`; no toca SQX ni el demonio.

Cómo se ejecuta

```
python3 -m portfolio.common.construct.candidates --project Test_USDJPY_donchianUpperCrossUp_H1
```

flag	obligatorio	qué hace
`--project`	no	un solo proyecto; sin él, todos los que tienen informes

Tarda unos segundos. No toca SQX.

Qué produce

Nada en disco: solo imprime. Por proyecto, una línea por candidata con su clase, su paso, sus palabras y la orden de archivo, y al final cuántas quedaron fuera y por qué palabra.

Cómo se lee el resultado

Las reglas son las tuyas (2026-09-30):

- **Superviviente**: la palabra del último paso que la juzgó es MANTENER, survives, worth_it, predicts o PASS.
- **Dudosa** (casi-superviviente): DUDOSA o INCONCLUSIVE. Entra al pool, marcada.
- **Lo que el workflow llevó adelante manda**: toda madre con lote de variantes en el 16.5 entra como superviviente del paso 16.5, digan lo que digan las palabras de antes. Su línea muestra **todas** esas palabras (también los DESCARTAR) para que lo veas al aprobarla.
- El FAIL de **mcRetest** no excluye: informa, no filtra (así lo hizo el workflow).
- Una estrategia ya archivada no se repite.

Una estrategia archivada con paso < 26 es **de desarrollo** automáticamente; no hay que marcarla.

Un ejemplo completo

Salida real del 2026-09-30, con el run H1 aún en el paso 16.5 (recortada: la lista entera tiene 170 líneas de candidatas):

```
=== Test_USDJPY_donchianUpperCrossUp_H1 ===
  [SUPERVIVIENTE] paso 16.5 · llevada al 16.5 · [crossTF control_failed, crossTF unusable,
crossmarket DESCARTAR, decay DUDOSA, edgeCost MANTENER, feedQuality MANTENER, gate MANTENER,
snoopingScreen MANTENER, spread MANTENER] · 032522d4cede…
    python3 -m core.archive archive --project Test_USDJPY_donchianUpperCrossUp_H1 --databank
Results --identity 032522d4cedef71ce06e3dbeaff3461a939f965ebb2530ea2999c88fcfbb782b --step 16.5
--family donchianUpperCrossUp
  [SUPERVIVIENTE] paso 8 · edgeCost · MANTENER · 003863ecc1d8…
    python3 -m core.archive archive --project Test_USDJPY_donchianUpperCrossUp_H1 --databank
Results --identity 003863ecc1d85d6b2f7b71d91be63f495c736fd80bbac5689693dfd4a4cbef45 --step 8 --
family donchianUpperCrossUp
  …
  170 supervivientes, 0 dudosas, 0 ya archivadas
  descartadas por «DESCARTAR»: 114
  descartadas por «control_failed»: 3
  descartadas por «fails»: 1
  descartadas por «inherited»: 11
  descartadas por «unusable»: 1
```

Las 14 primeras líneas son las madres que llegaron al 16.5; las otras 156 son supervivientes del paso 8 que el workflow no llevó más lejos.

Después, para formar el pool se declara con un nombre (un pool declarado queda **congelado**: otro pool es otro nombre). Eso lo hace hoy el motor desde Python (`inputs.pool.declare`); la ventana lo tendrá en la zona PORTFOLIOS.

Qué NO te dice

- **No te dice si una estrategia es buena**: solo que el workflow no la descartó hasta ese paso.
- **No sabe qué es un oos2 disponible**: el archivo guarda hoy build y oos1. Las madres del 16.5 tienen su curva de oos2 en el lote, pero sus operaciones de oos2 aún no existen en ningún export (2026-09-30) — por eso el veredicto de una cartera de desarrollo esperará a que el run termine.
- La etiqueta «paso 8 · edgeCost» nombra el primer estudio del paso: todos los del paso 8 dijeron MANTENER para esa estrategia.

Si algo falla

- `KeyError: 'strategy'` o `'identity'`: un `verdict.csv` antiguo sin esas columnas; se ignora el estudio que no tiene `identity`, pero uno con `identity` y sin `strategy` rompe: dilo.
- Una madre que esperabas no aparece: su lote en `strategyPermutations/<proyecto>/` aún no tiene carpeta `sqx/` (el 16.5 no ha llegado a ella).

78. Cartera · el universo y las parejas — la equity de cada estrategia, y qué parejas pueden ir juntas

Qué pregunta responde

Para un pool declarado: ¿cómo se movió la equity de cada estrategia **cada día y dentro del día** (flotante incluido, minuto a minuto desde M1), en el reloj de cada empresa de fondeo? ¿Cuadra con lo que dice el propio SQX? Y, con eso: ¿qué parejas de estrategias se parecen demasiado para ir en la misma cartera, y por qué filtro cae cada una?

Es el universo (hito M1) y el cribado de parejas (hito M2) del motor de carteras. No elige cartera todavía: eso es la búsqueda (F2 para fondeo), que espera a este paso.

Cuándo lo usas, y cuándo no

Úsalo cuando tengas un pool declarado (capítulo 77) y quieras ver si sus estrategias se pueden combinar. **No** lo uses para juzgar una estrategia sola: el flotante y la reconciliación son solo el punto de partida.

Antes de empezar

- Un pool declarado en `AlgoData/portfolio/pools/<nombre>.csv`.
- Las barras M1 del feed de cada estrategia en `AlgoData/bars/` (USDJPY y XAUUSD están; los índices no, los importarás cuando haga falta).
- No toca SQX. Lee un fichero de SQX solo para saber el huso de cada feed, en modo lectura.

Cómo se ejecuta

```
python3 -m portfolio.common.construct.universe --pool m1_test_uno
```

flag	obligatorio	qué hace
`--pool`	sí	el nombre del pool declarado
`--set`	no	cambia un ajuste de `portfolio/common/construct/config.yaml`, p. ej. `--set pairs.pearson=0.45`. Un umbral distinto del tuyo marca el run como **AJUSTADO** en la primera línea

Tarda unos 7 s por estrategia (lee el M1 de cada feed una sola vez) y usa ~2,6 GB de RAM. La segunda vez con el mismo pool y la misma configuración lee la caché y tarda nada.

Qué produce

Una carpeta por pool **y** configuración en `AlgoData/portfolio/universe/<hash del pool>-<hash de la config>/` (una caché: se puede borrar, se rehace):

fichero	qué es
`daily.parquet`	P&L diario a mercado de cada estrategia, en EET (tu reloj común), 0 dentro de su historia y vacío fuera
`monthly.parquet`	lo mismo por mes
`days_ftmo.parquet`, `days_hantec.parquet`	por día del servidor de cada empresa: lo cerrado, el flotante al cierre, el peor y el mejor momento del día
`m5_ftmo.npz`, `m5_hantec.npz`	el peor y el mejor flotante de cada bloque de 5 minutos, para sumar estrategias sin perder cuándo coinciden
`reconcile.csv`	cuánto cuadra cada estrategia con SQX
`pairs.parquet`	cada medida de cada pareja, solo sobre build
`manifest.json`	qué pool, qué config, qué relojes, qué versión del código

Cómo se lee el resultado

La reconciliación es la licencia para usar el flotante reconstruido. Dos pruebas contra SQX:

- **daily-low**: la curva diaria de SQX es, cada día, la equity **más baja** de ese día (lo cerrado más las posiciones abiertas en su peor mecha). El motor la reconstruye desde M1 y debe coincidir a menos de 1 $ en al menos el 99 % de los días. En la estrategia archivada coincide el 100 % (máx. 0,16 $).
- **MAE / MFE**: el peor y el mejor flotante de cada operación, contra las columnas `MAE ($)` y `MFE ($)` de SQX. En la archivada, 100 % y 99,9 %.

Una estrategia que no cuadra sale como **excluida**, con su motivo, y no entra en nada: nunca se usa en silencio.

Relojes sin confirmar: el del servidor de Hantec (Nueva York + 7 h) lo diste tú, sin confirmar; hasta leerlo del terminal MT5 de Hantec, todo lo de Hantec lo dice.

Las parejas (solo build, tus umbrales de 0,30):

- Pearson y Spearman, **mensual y diario**: cae si |ρ| > 0,30 en cualquiera.
- Co-loss (meses en que pierden las dos) y cola: caen si pasan de 0,30 **por arriba** (la cola entre dos estrategias independientes da −0,46 por cómo se eligen sus meses; por eso solo cuenta el lado positivo).
- Ventanas móviles de 60 meses (todo el build y las que acaban en sus últimos 36 meses): cae si el máximo |ρ| pasa de 0,30. Con menos de 60 meses compartidos no se aplican, y se cuenta.
- **Menos de 24 meses compartidos: fuera**, diga lo que diga el coeficiente.
- La correlación en días de estrés (el peor 5 % de días del pool) y el N efectivo **solo se informan**.

Un ejemplo completo

Salida real del 2026-09-30, pool de una sola estrategia (la archivada de USDJPY M30):

```
Universo del pool 'm1_test_uno' (c6a56a636ba5…)
Miembros: 1 conservados, 0 excluidos de 1
  4d679e0c2ce2… [conservada] daily-low build 100%, oos1 100% · MAE 100% · MFE 100%
Calendario: build 2008-01-01…2017-12-31, oos1 2018-01-01…2022-12-31, oos2 2023-01-01…2026-08-30
Relojes sin confirmar: ['hantec']
Carpeta:
/home/sergioguslw/Desktop/AlgoData/portfolio/universe/c6a56a636ba5d09b44a7dd1bb9ffa74c149fc4846
463e6ca9ebfae09d2b5dc0b-07f85832e838bf0d
Tiempo: 6.81 s · RAM pico: 2631 MB
```

Con una sola estrategia no hay parejas. Con varias, debajo sale el cribado; así se lee en un pool de prueba de tres, dos de ellas casi clones:

```
Parejas (solo build): 2 admisibles de 3; 3 de 3 estrategias tienen al menos una pareja
admisible
  caen por pearson_monthly: 1
  caen por pearson_daily: 1
  caen por spearman_monthly: 1
  …
N efectivo: calma 1.9 / 1.8, estrés 1.5 / 4.0 (autovalores / media de ρ)
```

Qué NO te dice

- **Que dos estrategias independientes pasen**: con tus umbrales, solo el 75 % de las parejas de estrategias **independientes** pasan todos los filtros sobre 10 años de build (las ventanas móviles y el co-loss rechazan por azar). Para K estrategias independientes, que pasen todas sus parejas es 0,75 elevado al número de parejas: 4 → 18 %, 6 → 1,5 %. Una cartera pequeña puede ser ruido de los filtros, no redundancia.
- **El swap de las noches**: SQX lo cobra al cerrar; el flotante de las noches no lo lleva.
- **El N efectivo por «media de ρ»** puede pasar de K cuando las correlaciones son negativas (4,0 con 3 estrategias arriba): léelo junto al de autovalores.
- Nada de oos1 ni oos2: las parejas se miden solo en build.

Si algo falla

- `excluida · motivo: …`: esa estrategia no cuadra con SQX; mírala antes de usarla.
- Un pool que no existe: `FileNotFoundError` con la ruta del CSV.
- Falta el M1 de un feed: `FileNotFoundError` en `AlgoData/bars/<feed>/M1.parquet`.

79. Cartera · la búsqueda de fondeo — qué cartera tiene más probabilidad de aprobar un plan

Qué pregunta responde

Para un plan de fondeo concreto (p. ej. Hantec Enhanced de 10k): de todas las carteras que el cribado de parejas permite, ¿cuál aprueba el challenge con más probabilidad en **6 meses** si la hubieras comprado cualquier día del build, y a qué riesgo por operación? Es la etapa A del camino de fondeo (hito F2 de `portfolio/PLAN.md`): elige candidatas mirando solo build. El juicio fuera de muestra (oos1 + oos2) y el Monte Carlo vienen después (F3, F4).

Cuándo lo usas, y cuándo no

Úsalo cuando tengas un pool cuyas estrategias llevan ya el **stop del paso 24** injertado. **No** te dice si la cartera es buena fuera de muestra, ni si el plan te sale rentable: eso es F4 y la economía del banco (F5).

Antes de empezar

- Un pool declarado (capítulo 77) cuyos miembros sean las versiones **con stop** del paso 24 (SL = X·ATR(20), X = p90), archivadas con su propio retest. Una estrategia sin ese stop queda **fuera** y se dice.
- Nada de SQX. Usa todos los núcleos de la máquina y hasta 80 GB de RAM: **no lo lances mientras el custodio corre un trabajo largo** (los dos no caben juntos).

Cómo se ejecuta

```
python3 -m portfolio.common.construct.funded --pool mi_pool --plan hantec:enhanced:10000:USD
```

flag	obligatorio	qué hace
`--pool`	sí	el pool declarado
`--plan`	sí	el plan, con la clave del catálogo (`python3 -m portfolio.funded.catalog.show hantec`)
`--set`	no	cambia un ajuste de `config.yaml` , p. ej. `--set funded.horizon_days=63`

Qué produce

`AlgoData/portfolio/runs/<fecha>_<pool>_<plan>/` : `search.parquet` (cada cartera evaluada, con sus miembros, su P(aprobar), su riesgo y de dónde salió) y `manifest.json` . Además, **filas en el ledger**: una en el estudio `PORTFOLIO_<pool>_construct` y una en el estudio de cada miembro, con todas las carteras probadas — cada una cuenta como un intento para el Sharpe deflactado.

Cómo se lee el resultado

Las reglas son las tuyas (2026-09-30):

- **Riesgo fijo**: cada operación arriesga r % del saldo **inicial** con su stop X·ATR(20). SQX simuló con 1.000 $ sobre 4·ATR(20); el motor lo convierte con una constante por estrategia.
- **Rejilla**: r = 0,1 %, 0,2 % … 1,0 %, igual para todas las estrategias de la cartera. Un nivel en el que alguna operación quedaría por debajo de 0,01 lotes no es factible.
- **P(aprobar)**: de todos los días de build en los que podrías haber comprado (con 6 meses por delante), qué parte aprueba todas las fases en ≤ 126 días de trading. Las reglas se miran sobre la equity **flotante** del día del servidor de la empresa, con el peor momento del día desde M1.
- **Mismo riesgo en todas las fases** aquí; el riesgo por fase se barre en la etapa B.

La salida lista las 10 mejores: `K` (cuántas estrategias), `P(pass)`, `r` y quiénes son. Una regla sin confirmar del catálogo o el reloj de Hantec salen nombrados.

Un ejemplo completo

Salida real del 2026-09-30 con el pool de prueba: su única estrategia no tiene el stop del paso 24, así que el comando se para sin buscar nada.

```
Pool 'm1_test_uno' plan hantec:enhanced:10000:USD: 1 miembros en el universo
  excluida 4d679e0c2ce2…: sin stop del paso 24
Ningun miembro utilizable: no hay stop del paso 24 en ninguno.
```

La primera búsqueda real espera a que el run H1 llegue al paso 24 y archives sus versiones con stop. Rendimiento medido en datos sintéticos (50 estrategias, 10 años): ~1.500 carteras por segundo en 48 núcleos, así que una búsqueda de 30.000 carteras tarda unos 20 s por plan.

Qué NO te dice

- **Nada fuera de muestra**: elige en build; P(aprobar) en build está inflada por la propia búsqueda.
- **No hay Monte Carlo** todavía: es el camino real de la historia desde cada día de inicio.
- **El redondeo de lotes** no se simula aquí (solo se comprueba que ninguno quede en cero).
- **Una cartera repetida por el genético cuenta dos veces** en el ledger: el número de intentos sale por exceso, nunca por defecto.

Si algo falla

- `sin stop del paso 24`: ese miembro es la versión original; archiva la versión con stop.
- `NotImplementedError` con `min_days`: el catálogo no sabe los días mínimos de ese plan; el fundingWatcher o tú tenéis que confirmarlo.

87. Cartera · la etapa B y el veredicto de fondeo — ¿aguanta la cartera elegida un Monte Carlo, y fuera de muestra?

Qué pregunta responde

La búsqueda de fondeo (capítulo 79) te deja miles de carteras ordenadas por su probabilidad de aprobar un plan mirando la historia real de build. Aquí se cogen **las mejores, distintas entre sí**, y se les hace la prueba dura: tres Monte Carlo distintos, con el riesgo **por fase**. Sale una ganadora por plan. Después, el veredicto: esa ganadora, leída en **oos1 + oos2**, ¿queda por encima de lo que saca una búsqueda igual hecha sobre carteras al azar del mismo pool?

Son los hitos F3 (etapa B) y F4 (veredicto) de `portfolio/PLAN.md` §14.

Cuándo lo usas, y cuándo no

Después de una búsqueda de fondeo (capítulo 79) que haya dejado su carpeta en `AlgoData/portfolio/runs/`. **No** lo uses para decidir cuánto dinero meter ni cuántas cuentas comprar: eso es la simulación del banco (F5). Y **no** emite veredicto si el universo no tiene oos2: el veredicto se juzga en oos1 y oos2 juntos, o no se emite.

Antes de empezar

- La carpeta de una búsqueda de fondeo ya hecha, con su `search.parquet` y su `manifest.json`.
- Las estrategias del pool, archivadas con su stop del paso 24, y las barras M1 de sus mercados.
- Nada de SQX. Usa todos los núcleos y hasta 80 GB: **no lo lances con el custodio trabajando**.
- **Dos ajustes no tienen valor por defecto y el comando se para si no los das** (son decisiones tuyas, todavía abiertas):
- `stageb.shared_of`: «no dos finalistas comparten más de la mitad de sus miembros» — ¿la mitad de cuál, cuando una cartera tiene 3 estrategias y la otra 6? `smaller` (de la pequeña) o `larger` (de la grande).
- `stageb.t1_gap`: cómo se separan las operaciones en el Monte Carlo T1. Solo hay una forma construida, `close_to_open_blocks` (ver abajo); se te pide que la aceptes a propósito.

Cómo se ejecuta

```
python3 -m portfolio.common.construct.funded_b \
  --run ~/Desktop/AlgoData/portfolio/runs/2026-10-02T1405_mi_pool_hantec_enhanced_10000_USD \
  --set stageb.shared_of=smaller --set stageb.t1_gap=close_to_open_blocks
```

flag	obligatorio	qué hace
`--run`	sí	la carpeta de la búsqueda de fondeo (etapa A) sobre la que se trabaja
`--set stageb.shared_of=…`	sí	`smaller` o `larger`, arriba
`--set stageb.t1_gap=close_to_open_blocks`	sí	acepta el modelo de huecos de T1
`--set stageb.shortlist_size=50`	no	cuántas finalistas (50)
`--set stageb.n_paths=10000`	no	caminos por Monte Carlo y longitud de bloque (10.000)
`--set verdict.null_draws=100`	no	cuántas búsquedas al azar forman el nulo (100)
`--set verdict.null_candidates=0`	no	carteras al azar por búsqueda nula; 0 = tantas como evaluó la etapa A

Si la búsqueda de la etapa A se lanzó con algún `--set` (por ejemplo otro horizonte), pásale aquí el mismo. No toca SQX.

Cuánto tarda (medido el 2026-10-02 en 8 hilos y con la máquina ocupada): una finalista de 10 estrategias, a tamaño completo (10.000 caminos × 11 juegos de caminos × 100 riesgos), 17 segundos; 50 finalistas, unos 14 minutos. **El veredicto repite la etapa B una vez por cada búsqueda nula**: con 100 son horas. Es el veredicto, no la etapa B, lo que pide la máquina entera.

Qué produce

En la misma carpeta de la búsqueda:

archivo	qué es
`shortlist.parquet`	una fila por finalista: sus miembros, su nota de la etapa A, su P(aprobar) de la etapa B, el riesgo por fase con que se queda, qué Monte Carlo decidió y con qué bloque, y la cifra de cada uno de los tres
`stageb.parquet`	**todas** las cifras: por finalista, Monte Carlo, longitud de bloque y riesgo — P(aprobar), P de cada fase, días hasta aprobar, qué regla rompe
`verdict.json`	la ganadora, la prueba de estabilidad, el veredicto (o por qué no se emite), cada búsqueda nula con sus cifras, la semilla y el coste
`verdict_rows.parquet`	las cifras de la ganadora en oos1 + oos2 (solo si hay veredicto)

Además, **filas en el ledger**: la etapa B una búsqueda por plan (finalistas × riesgos, todas cuentan como intentos); el veredicto, una por cada segmento que lee (oos1 y oos2). Cada fila va al estudio del pool y al de cada estrategia.

Cómo se lee el resultado

Las finalistas. De todas las carteras distintas de la etapa A se recorren de mejor a peor y se queda una si no comparte más de la mitad de sus miembros con ninguna de las ya elegidas.

Los tres Monte Carlo — cada uno fabrica 10.000 «semestres» posibles (126 días de servidor) y pasa las reglas del plan por cada uno:

nombre	qué sortea	qué conserva	qué pierde
D	rachas de días seguidos de la cartera entera	que las estrategias pierdan a la vez; el peor momento exacto de cada día (de M1)	una operación abierta al final de una racha se liquida ahí a su valor
T1	las operaciones de cada estrategia, con reposición, una tras otra con un hueco sorteado de su propia historia; cada estrategia por su cuenta	cada operación entera, con su flotante	que pierdan a la vez: lee menos caída de la real
T2	ventanas de calendario; todas las operaciones de todas las estrategias que se abrieron en la ventana, juntas	cada operación entera **y** qué se abrió a la vez	variedad: hay menos ventanas que operaciones

La longitud de las rachas no se elige, se mide: la que calcula Politis-White sobre el P&L diario de la propia cartera, y además 5, 10, 20 y 40 días. **Manda la más pesimista**, y entre los tres Monte Carlo **manda también el más pesimista** (tu regla del 2026-09-30). Todas las cifras se guardan.

El riesgo por fase. Cada fase del challenge prueba los 10 niveles de riesgo (0,1 % … 1,0 % por operación): un plan de dos fases son 100 combinaciones. La cartera se queda con la combinación cuya cifra pesimista es más alta.

Las cifras de cada fila:

- `P(aprobar)`: de los 10.000 caminos, cuántos aprueban todas las fases dentro del horizonte.
- `p_stage1`, `p_stage2`…: cuántos pasan la fase 1, cuántos la 2.
- `t_median`, `t_p90`: días de servidor desde la compra hasta aprobar, **solo entre los caminos que aprueban**. Los que llegan al final del horizonte sin aprobar ni romper van aparte, en `p_open`.
- `fail_daily_loss`, `fail_max_loss`, `fail_time_limit`: qué parte de los caminos rompe cada regla.

La estabilidad. La cifra que decide se recalcula 8 veces con sorteos nuevos. Si entre la mayor y la menor hay más de 2 puntos, lo dice: sube `stageb.n_paths` antes de decidir nada.

El veredicto. Se repite la misma búsqueda — misma nota de la etapa A, mismas finalistas diversas, misma etapa B — pero sobre **carteras admisibles al azar** del mismo pool, 100 veces. Cada búsqueda al azar da su propia ganadora. Luego la tuya y las 100 se leen en oos1 + oos2 con dos medidas, y se dice **en qué percentil queda la tuya**:

- *P(aprobar) en el camino real, desde cada día de inicio*: comprando el plan cada día de oos1 + oos2, qué parte de las veces aprueba.
- *P(aprobar) del Monte Carlo más pesimista*: lo mismo que la etapa B, con los datos de oos1 + oos2.

Y «¿se sostuvo la diversificación?»: las correlaciones de las parejas de la cartera, medidas otra vez en oos1 + oos2 contra los mismos umbrales. Solo se informa; no quita ni cambia nada.

No hay listón. Qué percentil aprueba es una decisión tuya que sigue abierta (Q11). Hasta que la fijes, el veredicto dice «SIN CRITERIO» y te da las cifras. Cuando la fijes, entra por `verdict.pass_measure` y `verdict.pass_percentile` y la última línea dirá PASA o NO PASA.

Un ejemplo completo

Hoy no hay ningún pool real con el stop del paso 24, así que no existe todavía una búsqueda real sobre la que correrlo. Esta es la salida de verdad sobre un pool **sintético** de 5 estrategias (el de `tests/test_portfolio_verdict.py`, 2026-10-02), con 3 búsquedas nulas y 400 caminos:

```
Etapa B · pool 'synthetic' plan test:x:10000:USD: 4 finalistas diversas de 20 carteras
distintas (20 evaluadas en la etapa A) × 9 riesgos
  K=2 A=0.742 B=0.708 r=1.0%/1.0% decide T2@1 · D 0.713 T1 0.755 T2 0.708 [m00, m01]
  K=3 A=0.923 B=0.698 r=1.0%/1.0% decide T2@2 · D 0.735 T1 0.752 T2 0.698 [m00, m02, m03]
  K=2 A=0.719 B=0.568 r=1.0%/1.0% decide T2@20 · D 0.605 T1 0.685 T2 0.568 [m01, m03]
  K=2 A=0.742 B=0.522 r=1.0%/1.0% decide T2@2 · D 0.557 T1 0.535 T2 0.522 [m01, m02]
Ganadora: P(aprobar) 0.708 con el Monte Carlo más pesimista (T2, bloque 1); tarda 21 días de
mediana (p90 36) entre las que aprueban; 18.2% siguen abiertas al acabar el horizonte; rompe
pérdida diaria 9.8%, pérdida máxima 1.2%, plazo 0.0%
Estabilidad: 3 repeticiones, dispersión 0.010 — los caminos sorteados bastan
La cartera de 2 estrategias (de 5 admisibles; 20 combinaciones probadas en la etapa A, 4
finalistas × 9 riesgos en la etapa B), a riesgo 1.0% / 1.0% por fase, leída en oos1+oos2 contra
3 búsquedas al azar de 20 combinaciones cada una:
  P(aprobar) en el camino real, desde cada día de inicio: 0.462 — percentil 17% del nulo
seleccionado (mediana nula 0.719, p95 0.719)
  P(aprobar) del Monte Carlo más pesimista: 0.417 — percentil 33% del nulo seleccionado
(mediana nula 0.583, p95 0.607)
  diversificación fuera de muestra: 0 de 1 parejas pasan de algún umbral, 1 sin el solape
mínimo para juzgarlas
Veredicto: SIN CRITERIO. Nadie ha fijado qué percentil aprueba (Q11, tuya): con estas cifras
decides tú si se compra el plan; no se descarta ni se aprueba nada.
```

Desde el 2026-10-04 la línea «Ganadora» da **dos** cifras: «P(aprobar) X en caminos nuevos (la búsqueda la eligió con Y, el mejor de muchos sorteos)». X es la que vale: se recalcula sobre caminos que la búsqueda no vio. Y es el máximo de muchas estimaciones con ruido y lee alto — unos 2 puntos cuando las finalistas se parecen. En `verdict.json`, X es `build_p` e Y es `selector_p`.

Cómo leerlo: `A` es la nota de la etapa A (historia real de build), `B` la de la etapa B (la más pesimista de los tres Monte Carlo). La cartera de 3 tenía la mejor nota A (0,923) y se queda segunda en B: la nota A estaba inflada por la búsqueda. `decide T2@1` = la cifra más baja la dio T2 con ventanas de 1 día. Y fuera de muestra la ganadora queda por **debajo** de la mediana de las búsquedas al azar (percentil 17 %): en este pool sintético, donde todas las estrategias son iguales por construcción, elegir no aportó nada — que es lo que tenía que salir.

Y si falta un ajuste obligatorio, se para sin leer nada:

```
stageb.shared_of no está fijado: elige entre ['smaller', 'larger'] con --set stageb.shared_of=
<valor>; no hay valor por defecto.
```

Qué NO te dice

- **El percentil no dice si las estrategias tienen ventaja.** El nulo sale del mismo pool, así que la ventaja del pool está también en el nulo. Dice si **la búsqueda eligió mejor que el azar entre carteras admisibles**. En un pool pequeño, una búsqueda al azar con muchas candidatas encuentra la misma cartera y el percentil sale ~50 % aunque la cartera sea buena (medido: 100 % de las búsquedas nulas eligieron la misma pareja, percentil 0,50).
- **La P(aprobar) de la etapa B sigue siendo de build**: está inflada por haber elegido. La que vale es la de oos1 + oos2.
- **T1 no conserva la hora**: cada operación cae donde la deje el hueco sorteado. Sirve para la suerte del orden, no para lo que pasa entre estrategias.
- **Una operación a caballo del primer o el último día del tramo** no entra en T1 ni T2 (sí en D).
- **El redondeo de lotes no se simula**; solo se descarta el riesgo en que algún lote quedaría en cero.
- **Nada de dinero**: ni precio del plan, ni recompras, ni cobros. Eso es el banco (F5).

Si algo falla

- `stageb.shared_of no está fijado` / `stageb.t1_gap no está fijado`: pásalos con `--set`.
- Termina con «Decisión: NINGUNA todavía — sin oos2 en el universo»: las estrategias del pool no tienen operaciones en oos2. La etapa B sí se ha escrito; el veredicto espera a esos datos.
- `T1: los miembros […] tienen menos de 2 operaciones`: una estrategia casi no opera en el tramo y no hay huecos que sortear; no puede entrar en una cartera de fondeo.
- Una estrategia que estaba en el pool desaparece («licencia MAE/MFE»): está archivada con el reloj del feed antiguo y las barras de hoy están en otro (cambio del 2026-10-01). Hay que volver a cosecharla sobre el feed nuevo.

88. La caja del fondeo — lo que sale de tu banco contra lo que vuelve, plan por plan

Qué pregunta responde

Si compro este challenge con esta cartera y a este riesgo, **¿entra más dinero en mi cuenta del banco por cobros del que sale por compras?** No mira la rentabilidad de la cuenta fondeada: mira la caja real. Cuenta cada compra (también las recompras tras cada suspenso), cada cobro con su reparto, y el reembolso de la cuota donde la empresa lo da:

```
VE_banco = lo que esperas cobrar − lo que esperas pagar
```

Para cada plan de tu universo (cuentas de 10k como mucho, en USD, de las empresas activas) te da una fila con el riesgo por fase que más rinde, la probabilidad de aprobar cada fase y de llegar al primer cobro, cuántas compras hacen falta, cuánto dura y cuánto paga una fondeada, el VE por compra, por dólar y por mes, **cuánto dinero tienes que adelantar antes del primer cobro**, cuánto edge puedes perder antes de que la caja pierda, y cuánto de todo eso lo ganaría un mono.

Cómo lo simula

Coge un camino largo de la cartera (años de días del servidor de la empresa, con la equity flotante de cada día) y **compra las cuentas una detrás de otra sobre ese mismo camino**: compra el día 0 → fases del challenge → fondeada → cobro cada N días → una regla la rompe → al día siguiente recompra… hasta que se acaba el camino o la caja. Así dos cuentas seguidas viven el mismo régimen de mercado y suspenden juntas, como pasa de verdad. Eso se repite sobre miles de caminos.

- **El presupuesto es un tope duro**: tu cuenta del banco tiene 1.000 €; cada cobro vuelve a ella; una compra sólo se hace si queda dinero para pagarla.
- **El recorte** `h` quita esa fracción del edge de la cartera (h = 50 % → la cartera rinde en vivo la mitad que en el OOS). La tabla se ordena con el `h` que tú digas y enseña la curva entera.
- **El mono** es la misma cartera sin edge: mismos días, misma volatilidad, deriva cero. Lo que gana el mono es lo que vale el plan como lotería; lo que aportan tus EAs es la **ventaja** sobre él.
- **Los add-ons** se evalúan como ΔVE: el VE con el add-on menos el VE sin él. El precio del add-on ya está descontado (se paga en cada compra).

Cuándo lo usas, y cuándo no

Cuando el motor de carteras ya ha elegido y juzgado una cartera fondeada y quieres saber qué plan comprarle. **No** elige la cartera (eso es del motor, capítulo 79) y **no decide por ti**: ordena los planes con el recorte y la caja que le das. El criterio de compra es tuyo.

No sirve con caminos que no vengan del pool validado de esa empresa, ni para comparar empresas cuyas reglas siguen sin confirmar sin leer antes la lista «Sin confirmar» que imprime al final.

Antes de empezar

No toca SQX ni MetaTrader. Necesita:

1. **Un conjunto de caminos** (`.npz`), que hoy fabrica la etapa B del motor de carteras. Lleva la tabla de días del servidor de cada camino (`closed` , `float_end` , `low` , `high` , `opened`) más dos datos: `unit_risk` (cuánto se arriesga por operación en esas cifras) y `days_per_year` .
2. **Tus decisiones**, que el comando no supone nunca. Sin ellas se niega y te dice cuál falta:

clave	qué es
`owner.eurusd`	el cambio EUR/USD del día (dólares por euro). No se descarga: lo das tú y queda escrito con el resultado
`owner.rebuy`	tras un suspenso: `same_plan` (recomprar el mismo plan mientras quede caja) o `none` (una sola cuenta)
`owner.haircuts`	los recortes `h` de la curva, p. ej. `[0, 0.25, 0.5, 0.75, 1]`
`owner.haircut_decides`	el recorte con el que se ordena la tabla
`owner.payout_cushion`	fracción del tamaño que dejas en la cuenta en cada cobro; 0 = retirar todo
`owner.tax_rate`	fracción de cada cobro que se va en impuestos
`owner.payout_fee`	fracción de cada cobro que se va en comisiones de pago y divisa
`owner.p_payout_denied`	probabilidad de que la empresa deniegue un cobro (la cuenta se acaba ahí)
`owner.p_firm_closes_year`	probabilidad de que la empresa cierre en un año
`gaps.hantec_fee_refund`	¿Hantec devuelve la cuota con el primer cobro? El catálogo no lo sabe
`gaps.hantec_endurance_payout_days`	cada cuántos días cobra una Endurance fondeada. El catálogo no lo trae
`gaps.hantec_express_floor_after_payout`	qué pasa con la pérdida máxima de la Express fondeada tras un cobro: `stays` (el retiro no la mueve: queda donde la dejó el máximo alcanzado) o `initial` (desde el primer retiro queda fija en el balance inicial)
`gaps.ftmo_1step_floor_after_payout`	lo mismo para la FTMO 1-Step fondeada

El presupuesto (`owner.budget_eur` , 1.000 €) ya está puesto: lo fijaste el 2026-09-29.

Cómo se ejecuta

```
python3 -m portfolio.funded.sim.run \
  --paths ~/Desktop/AlgoData/portfolio/paths/mi_cartera.npz \
  --study PORTFOLIO_abc123def456_construct --symbol USDJPY+XAUUSD --segment oos2 \
  --set owner.eurusd=1.17 --set owner.rebuy=same_plan \
  --set "owner.haircuts=[0, 0.25, 0.5, 0.75, 1]" --set owner.haircut_decides=0.5 \
  --set owner.payout_cushion=0.02 --set owner.tax_rate=0.19 --set owner.payout_fee=0.02 \
  --set owner.p_payout_denied=0.02 --set owner.p_firm_closes_year=0.05 \
  --set gaps.hantec_fee_refund=false --set gaps.hantec_endurance_payout_days=14 \
  --set gaps.hantec_express_floor_after_payout=initial --set
gaps.ftmo_1step_floor_after_payout=stays
```

(Las cifras de arriba son un ejemplo de forma, no una recomendación.)

flag	obligatorio	qué hace
`--paths`	sí (o `--days`)	el conjunto de caminos
`--days` + `--horizon --n-paths --mean-block --unit-risk --days-per-year`	alternativa	parte de una tabla de días y sortea los caminos en el momento, con el bootstrap por bloques de la etapa B
`--study --symbol --segment`	sí con caminos reales	el estudio del ledger donde se cuenta la búsqueda: cada plan × add-on × riesgo probado es una mirada
`--monkey`	no	un conjunto de caminos del mono hecho aparte (entradas al azar de verdad, con costes); si no, el mono es la cartera sin deriva
`--seed`	no	raíz de los sorteos de cobro denegado y cierre; sin él se sortea una y se imprime
`--out`	no	carpeta de salida
`--set risk.challenge=[…]`, `--set risk.funded=[…]`	no	la rejilla de riesgo por operación (fracción del balance inicial) de las fases y de la fondeada
`--set addons.max_set=2`	no	evaluar los add-ons también por parejas

Tarda segundos: unas 30-130 milésimas por configuración con 10.000 caminos de 1.250 días.

Qué produce

En `AlgoData/portfolio/bank/<fecha>_<caminos>/`: `table.parquet` (la tabla, un plan por fila), `addons.parquet`, `search.parquet` (todas las configuraciones probadas con su VE), `result.json` (lo mismo en el formato que pinta la ventana) y `manifest.json` (los caminos, la semilla, **el cambio EUR/USD usado**, la configuración entera y la fila del ledger). No sobrescribe nada.

Cómo se lee el resultado

```
Caja del fondeo — caminos «sintetico_edge_0.10R» (SINTÉTICOS): 2000 × 1250 días · recorte que ordena h = 0.5 · caja 1000 € a 1.17 $/€ · 1305 configuraciones

plan                        precio$   riesgo % f→fond  VE/compra   VE/$   VE/mes           [90 %]     mono  ventaja h equil.      h=0%    h=25%    h=50%    h=75%   h=100%
ftmo:2step:10000:USD         104.13          1/1 → 1      626.3   6.01     36.1      [35.0, 37.2]    425.4   1657.6    >100%    4549.8   3245.6   2083.0   1148.0    425.4
hantec:enhanced:10000:USD     99.00          1/1 → 1      587.5   5.93     33.8      [32.7, 34.9]    348.0   1601.0    >100%    4395.8   3098.9   1949.0   1037.6    348.0
hantec:enhancedx:10000:USD    99.00          1/1 → 1      396.0   4.00     29.0      [27.9, 30.1]    138.3   1537.0    >100%    4115.6   2813.4   1675.2    792.6    104.0
hantec:endurance:10000:USD    59.00        1/1/1 → 1      287.5   4.87     24.7      [23.7, 25.7]    130.6   1294.4    >100%    3648.1   2409.8   1425.0    659.0    130.6
hantec:enhanced:5000:USD      59.00          1/1 → 1      284.2   4.82     16.3      [15.8, 16.9]    128.9    814.0    >100%    2176.3   1523.4    943.0    481.3    128.9
hantec:enhancedx:5000:USD     59.00          1/1 → 1      188.5   3.19     13.8      [13.3, 14.4]     22.3    775.0      99%    2031.1   1374.0    797.4    347.1     -8.3
hantec:endurance:5000:USD     29.00        1/1/1 → 1      144.2   4.97     12.4      [11.9, 12.9]     68.8    646.2    >100%    1825.8   1207.0    715.0    332.5     68.8
ftmo:1step:10000:USD          92.43            1 → 1       34.2   0.37      5.7        [5.2, 6.3]    -98.0    429.4      72%    1285.5    771.9    331.4    -44.9   -338.3
hantec:express:2000:USD       39.00      0.25 → 0.25      -25.9  -0.66     -0.7      [-0.8, -0.7]    -78.2     35.0       9%      11.7    -19.7    -43.2    -60.2    -78.2
hantec:express:5000:USD       59.00      0.25 → 0.25      -26.3  -0.45     -0.8      [-0.8, -0.7]   -113.5     69.7      32%      96.7     15.8    -43.9    -82.6   -115.9
hantec:express:10000:USD      99.00      0.25 → 0.25      -33.6  -0.34     -1.0      [-1.1, -0.8]   -185.8    129.7      38%     226.6     63.6    -56.1   -131.8   -192.5

plan                          P fases P 1er cobro  compras →1er cobro cobros/fond  vida d  desembolso p50/p90/p99 $ tope caja  regla que más suspende
ftmo:2step:10000:USD           73%/81%         94%     3.33       1.75       12.12     341              104/417/625        0%  pérdida máxima (challenge) (52%)
hantec:enhanced:10000:USD      73%/81%         94%     3.32       1.75       10.34     342               99/396/594        0%  pérdida máxima (challenge) (52%)
hantec:enhancedx:10000:USD     68%/72%         89%     4.23       2.31        2.29     310              198/594/891        0%  pérdida máxima (challenge) (60%)
hantec:endurance:10000:USD   72%/73%/72%       90%     4.96       2.52        9.63     248              118/354/531        0%  pérdida máxima (challenge) (69%)
hantec:enhanced:5000:USD       73%/81%         94%     3.32       1.75       10.34     342               59/236/354        0%  pérdida máxima (challenge) (52%)
hantec:enhancedx:5000:USD      68%/72%         89%     4.23       2.31        2.29     310              118/354/531        0%  pérdida máxima (challenge) (60%)
hantec:endurance:5000:USD    72%/73%/72%       90%     4.96       2.52        9.63     248               58/174/261        0%  pérdida máxima (challenge) (69%)
ftmo:1step:10000:USD             51%           99%     9.69       2.26        1.81      55              185/462/832        0%  pérdida máxima (fondeada) (47%)
hantec:express:2000:USD          68%           46%     1.67       1.09        6.21     285                39/78/117        0%  pérdida máxima (fondeada) (55%)
hantec:express:5000:USD          68%           46%     1.67       1.09        6.21     285               59/118/177        0%  pérdida máxima (fondeada) (55%)
hantec:express:10000:USD         68%           46%     1.67       1.09        6.21     285               99/198/297        0%  pérdida máxima (fondeada) (55%)
```

Primera tabla, la economía: **precio** en dólares; **riesgo** por operación en cada fase y en la fondeada (el mejor de la rejilla); **VE/compra**, **VE/$** (por dólar gastado en cuotas) y **VE/mes** con su intervalo al 90 % — ese intervalo es sólo el error de simular, no la duda sobre tu edge; **mono** y **ventaja** (tu VE menos el del mono); **h**

equil., el recorte con el que la caja deja de ganar — «80 %» quiere decir que aguantas perder el 80 % del edge; `>100 %` que gana incluso sin edge; y el VE con cada recorte.

Segunda tabla, las probabilidades: de aprobar cada fase, de llegar a cobrar alguna vez, compras totales y compras hasta el primer cobro, cobros por cuenta fondeada y días que vive, y **el desembolso antes del primer cobro** (mediana / p90 / p99): el drawdown de tu caja. «Tope caja» es el % de simulaciones en que el presupuesto se agotó. La última columna es la regla que más cuentas rompe: la que hay que atacar.

Un VE positivo con `h = 0` y negativo con el `h` que has elegido no es una decisión: la frase final nombra esos planes.

Un ejemplo completo

Con caminos **sintéticos** (2.000 caminos de 1.250 días, un edge inventado de +0,10 R por día con cierre; no es ninguna cartera tuya) y una política **de ejemplo** — recomprar el mismo plan, colchón del 2 %, sin impuestos ni comisiones, suelo de la Express y de la FTMO 1-Step fijo en el balance inicial tras el primer retiro, Hantec sin reembolso:

```
Caja del fondeo — caminos «sintetico_edge_0.10R» (SINTÉTICOS): 2000 × 1250 días · recorte que
ordena h = 0.5 · caja 1000 € a 1.17 $/€ · 1305 configuraciones

plan                          precio$   riesgo % f→fond  VE/compra   VE/$   VE/mes
[90 %]     mono  ventaja h equil.      h=0%    h=25%    h=50%    h=75%   h=100%
ftmo:2step:10000:USD          104.13          1/1 → 1      626.3   6.01     36.1      [35.0,
37.2]    425.4   1657.6    >100%    4549.8   3245.6   2083.0   1148.0    425.4
hantec:enhanced:10000:USD      99.00          1/1 → 1      587.5   5.93     33.8      [32.7,
34.9]    348.0   1601.0    >100%    4395.8   3098.9   1949.0   1037.6    348.0
hantec:enhancedx:10000:USD     99.00          1/1 → 1      396.0   4.00     29.0      [27.9,
30.1]    138.3   1537.0    >100%    4115.6   2813.4   1675.2    792.6    104.0
hantec:endurance:10000:USD     59.00        1/1/1 → 1      287.5   4.87     24.7      [23.7,
25.7]    130.6   1294.4    >100%    3648.1   2409.8   1425.0    659.0    130.6
hantec:enhanced:5000:USD       59.00          1/1 → 1      284.2   4.82     16.3      [15.8,
16.9]    128.9    814.0    >100%    2176.3   1523.4    943.0    481.3    128.9
hantec:enhancedx:5000:USD      59.00          1/1 → 1      188.5   3.19     13.8      [13.3,
14.4]     22.3    775.0      99%    2031.1   1374.0    797.4    347.1     -8.3
hantec:endurance:5000:USD      29.00        1/1/1 → 1      144.2   4.97     12.4      [11.9,
12.9]     68.8    646.2    >100%    1825.8   1207.0    715.0    332.5     68.8
ftmo:1step:10000:USD           92.43            1 → 1       34.2   0.37      5.7        [5.2,
6.3]    -98.0    429.4      72%    1285.5    771.9    331.4    -44.9   -338.3
hantec:express:2000:USD        39.00       0.25 → 0.25      -25.9  -0.66     -0.7      [-0.8,
-0.7]    -78.2     35.0       9%      11.7    -19.7    -43.2    -60.2    -78.2
hantec:express:5000:USD        59.00       0.25 → 0.25      -26.3  -0.45     -0.8      [-0.8,
-0.7]   -113.5     69.7      32%      96.7     15.8    -43.9    -82.6   -115.9
…
Con h = 50%, una caja de 1000 € (1 € = 1.17 $) y la política dada: el VE_banco más alto es
ftmo:2step:10000:USD, riesgo 1/1 → 1 % por operación (fases → fondeada), VE +36.1 $/mes [2020,
2146] $ en el horizonte; sigue positivo con cualquier edge del OOS; ventaja sobre el mono
+1657.6 $. Positivos sólo con h = 0% y no con h = 50%: hantec:express:2000:USD,
hantec:express:5000:USD, hantec:express:10000:USD. Esto ordena, no decide: el criterio robusto
de compra (encargo 33 §3.5.6) es tuyo.
```

Tres cosas que este ejemplo enseña y conviene recordar. El mono **gana** en varios planes: una cuenta fondeada es una opción (cobras parte de lo bueno y no pagas lo malo), así que la pura volatilidad ya vale algo, y este mono no paga costes; por eso se mira la ventaja y no el VE a secas. La lectura del suelo tras el cobro **cambia el orden**: con el suelo fijo en el balance inicial la Express es negativa con h = 50 % en estos caminos; con la otra lectura (`stays`) y retirando todo era positiva. Y la frase final nombra los planes que sólo ganan con el edge completo.

Sin tus decisiones, se niega:

```
No simulo: faltan decisiones tuyas o datos que el catálogo no trae. Dalos con --set <clave>=
<valor>:
  owner.rebuy — qué se hace tras un suspenso: same_plan (recomprar el mismo plan mientras quede
caja) o none
  gaps.hantec_fee_refund — ¿hantec devuelve la cuota con el primer cobro? (true/false)
```

Qué NO te dice

- **No dice qué comprar.** Ordena con el `h` y la caja que le das.
- **El intervalo no incluye la duda sobre el edge**, sólo el error de simulación. La duda sobre el edge es la curva VE(h).
- **El mono por defecto no paga costes**: su VE es un techo de lo que vale el plan como lotería.
- **Reglas sin confirmar**: imprime cuáles usa tal como las lee el catálogo, tu lectura del suelo tras el cobro, y dos supuestos propios de la fondeada — que los días mínimos y la consistencia se cuentan desde el cobro anterior, y que el primer cobro llega un ciclo después de empezar la fondeada.
- **El lote mínimo no está**: en cuentas pequeñas el riesgo real va a saltos; aquí es continuo.
- **Add-ons que no cambian la simulación** (noticias, primer cobro a demanda) no se evalúan: sólo suben el precio aquí, aunque puedan cambiar qué estrategias entran en el pool.
- El riesgo elegido es el mejor de la rejilla: si sale siempre el extremo, la rejilla es corta.

Si algo falla

- `No simulo: faltan decisiones tuyas…` — falta una clave de la tabla de arriba; la nombra.
- `No simulo: estos caminos salen de datos reales…` — faltan `--study`, `--symbol`, `--segment`.
- `ModuleNotFoundError: portfolio.common.construct.mc` con `--days` — el generador de caminos de la etapa B aún no está; usa `--paths` con un conjunto ya hecho.

96. Cartera: la búsqueda por rangos — qué estrategias del pool van juntas, elegidas en build

Qué pregunta responde

De un pool de estrategias archivadas, **qué combinación forma la mejor cartera** mirando sólo el tramo `build`. «Mejor» no es una sola cifra: cada combinación admisible se mide con cuatro métricas de su curva diaria y se ordena en cada una; gana la que queda más arriba de media en las cuatro.

Las cuatro métricas, sobre la cartera a pesos iguales (todas las estrategias al mismo riesgo):

métrica	qué mide	mejor
RAR %	la pendiente anual de una recta ajustada a la curva de capital: cuánto crece al año, sin depender de dónde empieza o acaba	alta
R^3 (R-cubed)	el RAR dividido por la media de los cinco peores drawdowns y por lo que duraron: crecer mucho con caídas cortas y poco hondas	alta
gain-to-pain	el beneficio medio anual dividido por la media de los N peores drawdowns (N = años del tramo)	alta
P(año perdedor)	qué parte de los años pierde dinero	baja

La puntuación va de 0 a 1: es la media de los cuatro puestos (el de P(año perdedor), al revés). 1 sería ser la mejor en las cuatro a la vez. Decidido el 2026-10-04 por el Council, por delegación tuya (Q10 a).

Cuándo lo usas, y cuándo no

Cuando ya hay un pool declarado (capítulo 77) y quieres la cartera para la **cuenta real**. Para una cuenta de fondeo la búsqueda es otra (capítulo 79): allí manda la probabilidad de pasar el plan.

No sirve para decidir si la cartera vale: sólo elige en `build`. El juicio fuera de muestra (`oos1` contra 10.000 carteras al azar, y `oos2` una sola vez con el Sharpe deflactado) es el hito siguiente y **no está construido**. Una cartera salida de aquí no se opera.

Antes de empezar

- El pool declarado y sus estrategias archivadas **con el stop del paso 24** (SL = X·ATR(20)): sin él no hay riesgo fijo y la estrategia se queda fuera, con su nombre en el informe. Hoy las 30 madres archivadas de `Test_USDJPY_donchianUpperCrossUp_H1` no lo llevan.
- No toca SQX. Construir el universo de un pool nuevo usa mucha memoria (capítulo 78): no lo lances con el custodio a pleno rendimiento.
- Tienes que decir **cómo se cuenta el año perdedor**; no lleva valor por defecto: `calendar` (años naturales) o `rolling` (todas las ventanas de 252 días).

Cómo se ejecuta

```
python3 -m portfolio.common.construct.real --pool desarrollo --set real.losing_year=calendar
```

flag	obligatorio	qué hace
`--pool`	sí	nombre del pool declarado
`--set real.losing_year=`	sí	`calendar` o `rolling`; sin él el comando se niega a empezar
`--set real.rank_reference=seeds`	sólo si hay más de 100.000 combinaciones	con tantas no se prueban todas: busca un algoritmo genético y los puestos se toman contra sus carteras de partida
`--base <carpeta de una run>`	no	en vez de buscar, juzga si cada estrategia que quedó fuera **entraría** en la cartera de esa run; pide `--set real.increment=finite` o `marginal`
`--set pairs.pearson=0.45` (y demás)	no	relaja un filtro de parejas; la run queda marcada como relajada

Cuántas estrategias caben no se teclea: lo fija la economía del fondeo (decisión del 2026-10-04). Una estrategia cuyo trade más pequeño no llega a 0,01 lotes en una cuenta de 10.000 ni arriesgando el 1 % por operación se queda fuera; con las demás, la cartera puede tener de 1 a tantas como quepan sin que ninguna pareja incumpla los filtros de correlación.

La búsqueda en sí tarda segundos; el universo, lo que diga el capítulo 78.

Qué produce

En `AlgoData/portfolio/runs/<fecha>_<pool>_real/`:

- `search.parquet` — una fila por combinación evaluada: miembros, K, puntuación, las cuatro métricas, el Sharpe diario y el riesgo mínimo por operación.
- `manifest.json` — la cartera elegida, cuántas se probaron, las excluidas y por qué. `verdict` vacío.
- Con `--base`: `admission.csv` dentro de la carpeta de esa run.

Y en el registro (ledger), paso 27, tramo `build`: **cada combinación evaluada cuenta como un intento**, en el estudio del pool y en el de cada estrategia. Ese recuento es el que luego rebaja el Sharpe de la cartera en `oos2`.

Cómo se lee el resultado

Todavía no hay captura de una run real: ningún pool tiene estrategias con el stop del paso 24. La salida tiene esta forma (las líneas las escribe `real.py`; las cifras salen del pool):

```
Pool '<pool>': N miembros en el universo, M utilizables
  excluida <identidad>…: sin stop del paso 24
Búsqueda exhaustive: C combinaciones admisibles evaluadas en build; rangos tomados entre C
CARTERA ELEGIDA: K=… [miembros] · puntuación … (0-1, media de los rangos de las 4 métricas) ·
deja fuera … estrategias
  al lote mínimo en 10,000: … % por operación y miembro, … % del balance arriesgado a la vez si
coinciden los K
  RAR … %/año · R³ … · gain-to-pain … · P(año perdedor) … · Sharpe diario …
Sin veredicto: es la elección en build; el juicio en oos1 y oos2 es el hito M5.
```

- **«deja fuera N estrategias»** es el coste de la elección, al lado de lo que gana.
- **«arriesgado a la vez»** es K por el riesgo mínimo de cada miembro: lo que la cuenta de 10.000 tiene en juego si todas abren a la vez al lote mínimo. Nadie ha fijado un tope para esa cifra.
- La puntuación **sólo se compara dentro de una misma run** (es un puesto entre las carteras de esa run). Entre runs se comparan las cuatro métricas.

Un ejemplo completo

La prueba `tests/test_portfolio_ranks.py` lo hace con cuatro estrategias inventadas: A y B ganan lo mismo y sus vaivenes son opuestos; C y D ganan menos. Sólo son admisibles las parejas A-B, A-C y C-D, así que hay 7 carteras posibles (4 sueltas y 3 parejas). Salida real:

```
OK     exhaustive: 4 singles + 3 admissible pairs, no triangle -> 7 combinations
OK     the engine's scores equal the slow independent computation
OK     the best portfolio is the hedged pair {A, B}
       {A,B}: score 0.857 · second best 0.821 · 7 combinaciones
```

Gana A+B porque al juntarlas los vaivenes se anulan: es la primera en R^3 y en gain-to-pain, empata en la cabeza en P(año perdedor) y es la segunda en RAR.

Qué NO te dice

- **No dice que la cartera funcione.** Está elegida mirando `build`; la mejor de miles de combinaciones siempre se ve bien en el tramo donde se eligió.
- No compara con carteras al azar: eso se hace sólo fuera de muestra, nunca en `build`.
- Los drawdowns son de la curva diaria (flotante al cierre de cada día incluido), **no del peor momento dentro del día**. Para fondeo, donde manda el flotante intradía, usa el capítulo 79.
- La admisión (`--base`) juzga cada candidata contra la misma cartera; no dice qué pasa si entran dos.

Si algo falla

- `real.losing_year no tiene valor y no lleva defecto` — falta el `--set`; elige una lectura.
- `Ningún miembro utilizable` — ninguna estrategia del pool lleva el stop del paso 24, o a ninguna le llega el lote mínimo.
- `real.rank_reference no tiene valor` — el pool tiene más de 100.000 combinaciones admisibles y hay que añadir `--set real.rank_reference=seeds`.